目录

0、简介
一、四元数的定义
二、欧拉角到四元数的转换
2.1 公式:
2.2 code:
三、四元数到欧拉角的转换
3.1 公式
3.2 code:
3.3 四元素到旋转矩阵转换
四. 奇点
五. 矢量旋转
证明:
六 . 其他参考
七 python 转换
7.1 四元素欧拉角互相转换
7.2 旋转矩阵<->欧拉角 py
八 Eigen transform
8.1 欧拉角到四元素
四元素得到yaw
四元素到旋转向量
旋转轴向量到四元素
Eigen 转换函数
九 旋转矩阵与欧拉角

0、简介

四元数与欧拉角之间的转换

百度百科四元素

在3D图形学中，最常用的旋转表示方法便是四元数和 欧拉角 ，比起矩阵来具有节省存储空间和方便插值的优点。

本文主要归纳了两种表达方式的转换，计算公式采用3D笛卡尔坐标系：

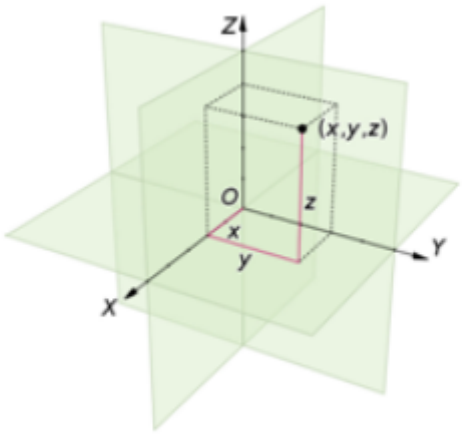


图1 3D Cartesian coordinate System (from wikipedia)

定义ψ，θ，ϕ分别为绕Z轴、Y轴、X轴的旋转角度，如果用Tait-Bryan angle表示，分别为Yaw、Pitch、Roll。

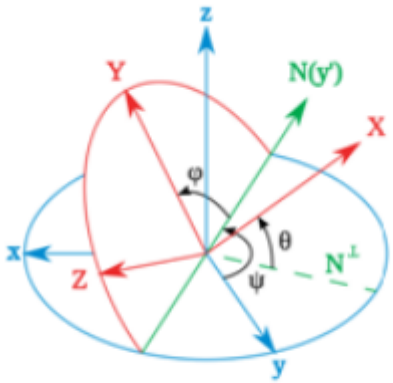


图2 Tait-Bryan angles (from wikipedia)

一、四元数的定义

$$q = [w, x, y, z]^T$$

$$|q|^2 = w^2 + x^2 + y^2 + z^2 = 1$$

$$\mathbf{q} = [\, q_0 \quad q_1 \quad q_2 \quad q_3 \,]^T = [\, q_w \quad q_x \quad q_y \quad q_z \,]^T$$
$$|\mathbf{q}|^2 = q_0^2 + q_1^2 + q_2^2 + q_3^2 = q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$$

- 通过旋转轴和绕该轴旋转的角度可以构造一个四元数：

$$w = cos(\alpha/2)$$

$$x = sin(\alpha/2)cos(\beta_x)$$

$$y = sin(\alpha/2)cos(\beta_y)$$

$$z = sin(\alpha/2)cos(\beta_z)$$

- 其中α是一个简单的旋转角（旋转角的弧度值），而$cos(\beta_x), cos(\beta_y), cos(\beta_z)$是定位旋转轴的“方向余弦”（欧拉旋转定理）。

利用欧拉角也可以实现一个物体在空间的旋转，它按照既定的顺序，如依次绕z,y,x分别旋转一个固定角度，使用yaw，pitch，roll分别表示物体绕,x,y,z的旋转角度，记为ψ，θ，ϕ，可以利用三个四元数依次表示这三次旋转，即：

$$Q_1 = cos(\psi/2) + sin(\psi/2)k$$

$$Q_2 = cos(\theta/2) + sin(\theta/2)j$$

$$Q_3 = cos(\phi/2) + sin(\phi/2)i$$

二、欧拉角到四元数的转换

2.1 公式：

$$\begin{aligned}
\mathbf{q}_{\mathrm{IB}} &= \begin{bmatrix} \cos(\psi/2) \\ 0 \\ 0 \\ \sin(\psi/2) \end{bmatrix} \begin{bmatrix} \cos(\theta/2) \\ 0 \\ \sin(\theta/2) \\ 0 \end{bmatrix} \begin{bmatrix} \cos(\phi/2) \\ \sin(\phi/2) \\ 0 \\ 0 \end{bmatrix} \\
&= \begin{bmatrix} \cos(\phi/2)\cos(\theta/2)\cos(\psi/2) + \sin(\phi/2)\sin(\theta/2)\sin(\psi/2) \\ \sin(\phi/2)\cos(\theta/2)\cos(\psi/2) - \cos(\phi/2)\sin(\theta/2)\sin(\psi/2) \\ \cos(\phi/2)\sin(\theta/2)\cos(\psi/2) + \sin(\phi/2)\cos(\theta/2)\sin(\psi/2) \\ \cos(\phi/2)\cos(\theta/2)\sin(\psi/2) - \sin(\phi/2)\sin(\theta/2)\cos(\psi/2) \end{bmatrix}
\end{aligned}$$

2.2 code:

```
struct Quaternion
{
    double w, x, y, z;
};

Quaternion ToQuaternion(double yaw, double pitch, double roll) // yaw (Z), pitch (Y)
{
    // Abbreviations for the various angular functions
    double cy = cos(yaw * 0.5);
    double sy = sin(yaw * 0.5);
    double cp = cos(pitch * 0.5);
    double sp = sin(pitch * 0.5);
    double cr = cos(roll * 0.5);
    double sr = sin(roll * 0.5);

    Quaternion q;
    q.w = cy * cp * cr + sy * sp * sr;
    q.x = cy * cp * sr - sy * sp * cr;
```

```
    q.y = sy * cp * sr + cy * sp * cr;
    q.z = sy * cp * cr - cy * sp * sr;

    return q;
}
```

三、四元数到欧拉角的转换

3.1 公式

可以从四元数通过以下关系式获得欧拉角：

$$\begin{bmatrix}\phi\\ \theta\\ \psi\end{bmatrix}=\begin{bmatrix}\arctan\frac{2(q_0q_1+q_2q_3)}{1-2(q_1^2+q_2^2)}\\ \arcsin(2(q_0q_2-q_3q_1))\\ \arctan\frac{2(q_0q_3+q_1q_2)}{1-2(q_2^2+q_3^2)}\end{bmatrix}$$

- arctan和arcsin的结果是$[-\frac{\pi}{2},\frac{\pi}{2}]$，这并不能覆盖所有朝向(对于$\theta$角$[-\frac{\pi}{2},\frac{\pi}{2}]$的取值范围已经满足)，因此需要用atan2来代替arctan。

$$\begin{bmatrix}\phi\\ \theta\\ \psi\end{bmatrix}=\begin{bmatrix}\mathrm{atan2}(2(q_0q_1+q_2q_3),1-2(q_1^2+q_2^2))\\ \mathrm{asin}(2(q_0q_2-q_3q_1))\\ \mathrm{atan2}(2(q_0q_3+q_1q_2),1-2(q_2^2+q_3^2))\end{bmatrix}$$

3.2 code:

```
#define _USE_MATH_DEFINES
#include <cmath>

struct Quaternion {
    double w, x, y, z;
};

struct EulerAngles {
    double roll, pitch, yaw;
};

EulerAngles ToEulerAngles(Quaternion q) {
    EulerAngles angles;
```

```
    // roll (x-axis rotation)
    double sinr_cosp = 2 * (q.w * q.x + q.y * q.z);
    double cosr_cosp = 1 - 2 * (q.x * q.x + q.y * q.y);
    angles.roll = std::atan2(sinr_cosp, cosr_cosp);

    // pitch (y-axis rotation)
    double sinp = 2 * (q.w * q.y - q.z * q.x);
    if (std::abs(sinp) >= 1)
        angles.pitch = std::copysign(M_PI / 2, sinp); // use 90 degrees if out of ra
    else
        angles.pitch = std::asin(sinp);

    // yaw (z-axis rotation)
    double siny_cosp = 2 * (q.w * q.z + q.x * q.y);
    double cosy_cosp = 1 - 2 * (q.y * q.y + q.z * q.z);
    angles.yaw = std::atan2(siny_cosp, cosy_cosp);

    return angles;
}
```

3.3 四元素到旋转矩阵 转换

$$R = \begin{bmatrix} 1-2(q_2^2+q_3^2) & 2(q_1q_2-q_0q_3) & 2(q_0q_2+q_1q_3) \\ 2(q_1q_2+q_0q_3) & 1-2(q_1^2+q_3^2) & 2(q_2q_3-q_0q_1) \\ 2(q_1q_3-q_0q_2) & 2(q_0q_1+q_2q_3) & 1-2(q_1^2+q_2^2) \end{bmatrix}$$

或等效地，通过齐次表达式：

$$R = \begin{bmatrix} q_0^2+q_1^2-q_2^2-q_3^2 & 2(q_1q_2-q_0q_3) & 2(q_0q_2+q_1q_3) \\ 2(q_1q_2+q_0q_3) & q_0^2-q_1^2+q_2^2-q_3^2 & 2(q_2q_3-q_0q_1) \\ 2(q_1q_3-q_0q_2) & 2(q_0q_1+q_2q_3) & q_0^2-q_1^2-q_2^2+q_3^2 \end{bmatrix}$$

四. 奇点

当螺距接近±90°（南北极）时，必须意识到欧拉角参数化的奇异性。这些情况必须特别处理。这种情况的通用名称是万向节锁。

处理奇异点的代码可从以下网站获取：www.euclideanspace.com

五. 矢量旋转

定义四元素的尺度q_0和向量$\vec{q}$，有$\mathbf{q} = (q_0, \vec{q}) = q_0 + iq_1 + jq_2 + kq_3$.

请注意，通过定义欧拉旋转的四元数q来旋转三维矢量$\vec{v}$的规范方法是通过公式：

$$\mathbf{p}' = \mathbf{q}\mathbf{p}\mathbf{q}^*$$

这儿：$\mathbf{p} = (0, \vec{v}) = 0 + iv_1 + jv_2 + kv_3$是包含嵌入向量$\vec{v}$的四元数，$\mathbf{q}^* = (q_0, -\vec{q})\mathbf{q}^* = (q_0, -\vec{q})$是共轭四元数，

在计算实现中，这需要两个四元数乘法。一种替代方法是应用一对关系：

$$\vec{t} = 2\vec{q} \times \vec{v}$$

$$\vec{v}' = \vec{v} + q_0\vec{t} + \vec{q} \times \vec{t}$$

×：表示三维矢量叉积。这涉及较少的乘法，因此计算速度更快。数值测试表明，对于矢量旋转，后一种方法可能比原始方法快30%[4]。

证明：

标量和矢量部分的四元数乘法的一般规则由下式给出：

$$\begin{aligned}\mathbf{q_1q_2} &= (r_1, \vec{v}_1)(r_2, \vec{v}_2) \\ &= (r_1r_2 - \vec{v}_1 \cdot \vec{v}_2, r_1\vec{v}_2 + r_2\vec{v}_1 + \vec{v}_1 \times \vec{v}_2)\end{aligned}$$

利用这种关系$\mathbf{p} = (0, \vec{v})$可以找到：

$$\begin{aligned}\mathbf{p}\mathbf{q}^* &= (0, \vec{v})(q_0, -\vec{q}) \\ &= (\vec{v} \cdot \vec{q}, q_0\vec{v} - \vec{v} \times \vec{q})\end{aligned}$$

并替换为三乘积：

$$\begin{aligned}\mathbf{q}\mathbf{p}\mathbf{q}^* &= (q_0, \vec{q})(\vec{v} \cdot \vec{q}, q_0\vec{v} - \vec{v} \times \vec{q}) \\ &= (0, q_0^2\vec{v} + q_0\vec{q} \times \vec{v} + (\vec{v} \cdot \vec{q})\vec{q} + q_0\vec{q} \times \vec{v} + \vec{q} \times (\vec{q} \times \vec{v}))\end{aligned}$$

$$q_0^2 = 1 - \vec{q} \cdot \vec{q}q_0^2 = 1 - \vec{q} \cdot \vec{q}$$

$$\vec{q} \times (\vec{q} \times \vec{v}) = (\vec{q} \cdot \vec{v})\vec{q} - (\vec{q} \cdot \vec{q})\vec{v}$$

可得到：

$$\mathbf{p}' = \mathbf{q}\mathbf{p}\mathbf{q}^* = (0, \vec{v} + 2q_0\vec{q} \times \vec{v} + 2\vec{q} \times (\vec{q} \times \vec{v}))$$

在定义$\vec{t} = 2\vec{q} \times \vec{v}$时，可以按标量和矢量部分来表示：

$$(0, \vec{v}') = (0, \vec{v} + q_0\vec{t} + \vec{q} \times \vec{t}).$$

六 . 其他参考

- Rotation operator (vector space)
- Quaternions and spatial rotation
- Euler Angles
- Rotation matrix

- Rotation formalisms in three dimensions

七 python 转换

7.1 四元素欧拉角互相转换

```
def EulerAndQuaternionTransform( intput_data):
    """
        四元素与欧拉角互换
    """
    data_len = len(intput_data)
    angle_is_not_rad = False

    if data_len == 3:
        r = 0
        p = 0
        y = 0
        if angle_is_not_rad: # 180 ->pi
            r = math.radians(intput_data[0])
            p = math.radians(intput_data[1])
            y = math.radians(intput_data[2])
        else:
            r = intput_data[0]
            p = intput_data[1]
            y = intput_data[2]

        sinp = math.sin(p/2)
        siny = math.sin(y/2)
        sinr = math.sin(r/2)

        cosp = math.cos(p/2)
        cosy = math.cos(y/2)
        cosr = math.cos(r/2)

        w = cosr*cosp*cosy + sinr*sinp*siny
        x = sinr*cosp*cosy - cosr*sinp*siny
        y = cosr*sinp*cosy + sinr*cosp*siny
        z = cosr*cosp*siny - sinr*sinp*cosy
        return [w,x,y,z]

    elif data_len == 4:

        w = intput_data[0]
        x = intput_data[1]
        y = intput_data[2]
        z = intput_data[3]
```

```
        r = math.atan2(2 * (w * x + y * z), 1 - 2 * (x * x + y * y))
    p = math.asin(2 * (w * y - z * x))
    y = math.atan2(2 * (w * z + x * y), 1 - 2 * (y * y + z * z))

    if angle_is_not_rad : # pi -> 180
        r = math.degrees(r)
        p = math.degrees(p)
        y = math.degrees(y)
    return [r,p,y]
```

7.2 旋转矩阵<->欧拉角 py

```
import numpy as np
import math
import random

def isRotationMatrix(R) :
    Rt = np.transpose(R)
    shouldBeIdentity = np.dot(Rt, R)
    I = np.identity(3, dtype = R.dtype)
    n = np.linalg.norm(I - shouldBeIdentity)
    return n < 1e-6

def rotationMatrixToEulerAngles(R) :

    assert(isRotationMatrix(R))
    sy = math.sqrt(R[0,0] * R[0,0] + R[1,0] * R[1,0])
    singular = sy < 1e-6

    if not singular :
        x = math.atan2(R[2,1] , R[2,2])
        y = math.atan2(-R[2,0], sy)
        z = math.atan2(R[1,0], R[0,0])
    else :
        x = math.atan2(-R[1,2], R[1,1])
        y = math.atan2(-R[2,0], sy)
        z = 0

    return np.array([x, y, z])

def eulerAnglesToRotationMatrix(theta) :

    R_x = np.array([[1,     0,         0          ],
    [0,     math.cos(theta[0]), -math.sin(theta[0]) ],
```

```
[0,     math.sin(theta[0]), math.cos(theta[0]) ]
                                                ])

R_y = np.array([[math.cos(theta[1]),  0,   math.sin(theta[1]) ],
[0,           1,   0          ],
[-math.sin(theta[1]),  0,   math.cos(theta[1]) ]
])

R_z = np.array([[math.cos(theta[2]),  -math.sin(theta[2]),  0],
[math.sin(theta[2]),  math.cos(theta[2]),   0],
[0,           0,           1]
])


R = np.dot(R_z, np.dot( R_y, R_x ))

return R
```

c++:

```
    static Eigen::Vector3d R2ypr(const Eigen::Matrix3d &R)
{
    Eigen::Vector3d n = R.col(0);
    Eigen::Vector3d o = R.col(1);
    Eigen::Vector3d a = R.col(2);

    Eigen::Vector3d ypr(3);
    double y = atan2(n(1), n(0));
    double p = atan2(-n(2), n(0) * cos(y) + n(1) * sin(y));
    double r = atan2(a(0) * sin(y) - a(1) * cos(y), -o(0) * sin(y) + o(1) * cos(
    ypr(0) = y;
    ypr(1) = p;
    ypr(2) = r;

    return ypr / M_PI * 180.0;
}

template <typename Derived>
static Eigen::Matrix<typename Derived::Scalar, 3, 3> ypr2R(const Eigen::MatrixBa
{
    typedef typename Derived::Scalar Scalar_t;

    Scalar_t y = ypr(0) / 180.0 * M_PI;
    Scalar_t p = ypr(1) / 180.0 * M_PI;
    Scalar_t r = ypr(2) / 180.0 * M_PI;
```

```
    Eigen::Matrix<Scalar_t, 3, 3> Rz;
    Rz << cos(y), -sin(y), 0,
        sin(y), cos(y), 0,
        0, 0, 1;

    Eigen::Matrix<Scalar_t, 3, 3> Ry;
    Ry << cos(p), 0., sin(p),
        0., 1., 0.,
        -sin(p), 0., cos(p);

    Eigen::Matrix<Scalar_t, 3, 3> Rx;
    Rx << 1., 0., 0.,
        0., cos(r), -sin(r),
        0., sin(r), cos(r);

    return Rz * Ry * Rx;
}
```

八 Eigen transform

8.1 欧拉角到四元素

```
Eigen::Quaterniond RollPitchYaw(const double roll, const double pitch,
                                const double yaw) {
  const Eigen::AngleAxisd roll_angle(roll, Eigen::Vector3d::UnitX());
  const Eigen::AngleAxisd pitch_angle(pitch, Eigen::Vector3d::UnitY());
  const Eigen::AngleAxisd yaw_angle(yaw, Eigen::Vector3d::UnitZ());
  return yaw_angle * pitch_angle * roll_angle;
}
```

四元素得到yaw

```
template <typename T>
  T GetYaw(const Eigen::Quaternion<T>& rotation) {
    const Eigen::Matrix<T, 3, 1> direction =
      rotation * Eigen::Matrix<T, 3, 1>::UnitX();
    return atan2(direction.y(), direction.x());
  }
```

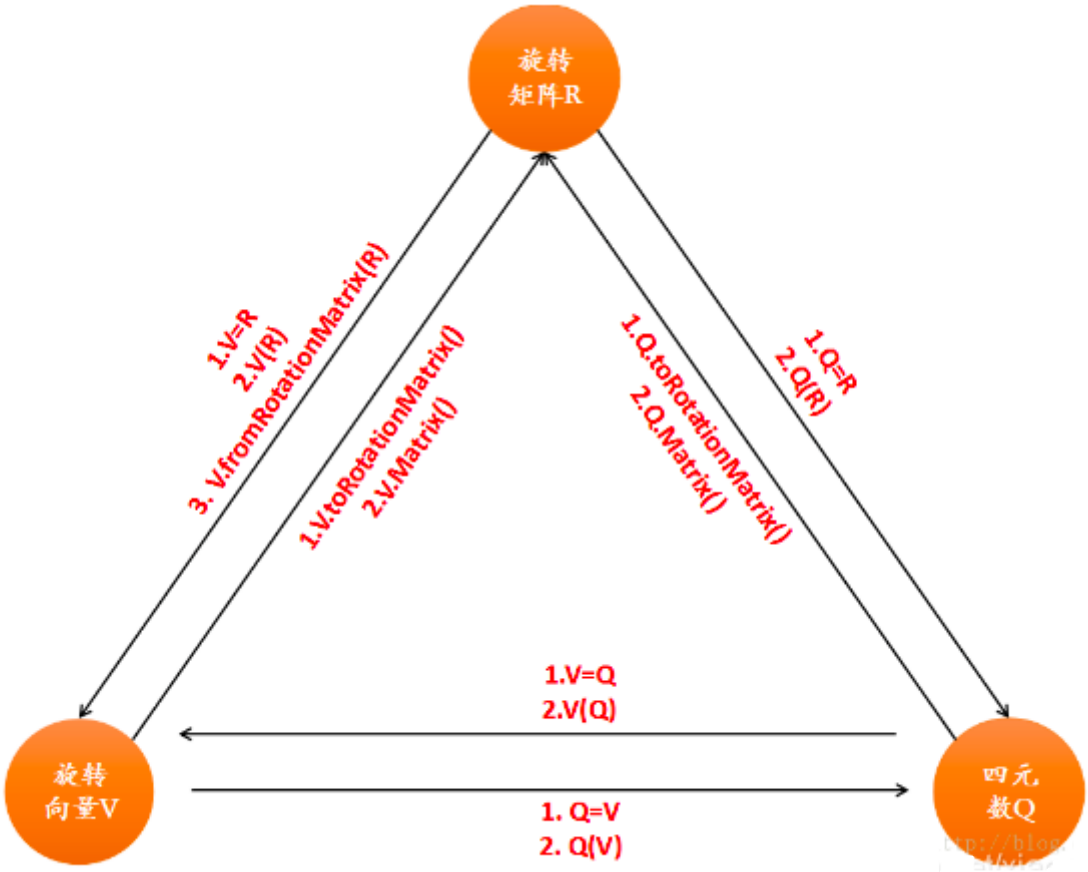
四元素到旋转向量

```
template <typename T>
  Eigen::Matrix<T, 3, 1> RotationQuaternionToAngleAxisVector(
    const Eigen::Quaternion<T>& quaternion) {
    Eigen::Quaternion<T> normalized_quaternion = quaternion.normalized();
    // We choose the quaternion with positive 'w', i.e., the one with a smaller
    // angle that represents this orientation.
    if (normalized_quaternion.w() < 0.) {
      // Multiply by -1. http://eigen.tuxfamily.org/bz/show_bug.cgi?id=560
      normalized_quaternion.w() = -1. * normalized_quaternion.w();
      normalized_quaternion.x() = -1. * normalized_quaternion.x();
      normalized_quaternion.y() = -1. * normalized_quaternion.y();
      normalized_quaternion.z() = -1. * normalized_quaternion.z();
    }
    // We convert the normalized_quaternion into a vector along the rotation axi
    // with length of the rotation angle.
    const T angle =
      2. * atan2(normalized_quaternion.vec().norm(), normalized_quaternion.w());
    constexpr double kCutoffAngle = 1e-7;  // We linearize below this angle.
    const T scale = angle < kCutoffAngle ? T(2.) : angle / sin(angle / 2.);
    return Eigen::Matrix<T, 3, 1>(scale * normalized_quaternion.x(),
                                  scale * normalized_quaternion.y(),
                                  scale * normalized_quaternion.z());
  }
```

旋转轴向量到四元素

```
template <typename T>
  Eigen::Quaternion<T> AngleAxisVectorToRotationQuaternion(
    const Eigen::Matrix<T, 3, 1>& angle_axis) {
    T scale = T(0.5);
    T w = T(1.);
    constexpr double kCutoffAngle = 1e-8;  // We linearize below this angle.
    if (angle_axis.squaredNorm() > kCutoffAngle) {
      const T norm = angle_axis.norm();
      scale = sin(norm / 2.) / norm;
      w = cos(norm / 2.);
    }
    const Eigen::Matrix<T, 3, 1> quaternion_xyz = scale * angle_axis;
    return Eigen::Quaternion<T>(w, quaternion_xyz.x(), quaternion_xyz.y(),
                                quaternion_xyz.z());
  }
```

Eigen 转换函数



九 旋转矩阵与欧拉角

按旋转坐标系分 内旋(旋转的轴是动态的) 和 外旋(旋转轴是固定的，是不会动的)。

绕定轴 XYZ旋转（RPY）（外旋）

- 假设两个坐标系A和B，二者初始时完全重合。 过程如下：B绕A的X轴旋转γ角，再绕A的Y轴旋转β角，最后绕A的Z轴旋转α角，完成旋转。整个过程，A不动B动。旋转矩阵的计算方法如下：R = Rz * Ry *Rx，乘法顺序：从右向左，依次旋转X轴Y轴Z轴 。

因为三次旋转都是相对于固定坐标系$\{A\}$而言的，按照“从右向左”的原则，得相应的旋转矩阵

$$
{}^{A}_{B}R_{xyz}(\gamma,\beta,\alpha)=R(z_A,\alpha)R(y_A,\beta)R(x_A,\gamma) \tag{2.36}
$$

$$
=\begin{bmatrix} c\alpha & -s\alpha & 0 \\ s\alpha & c\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} c\beta & 0 & s\beta \\ 0 & 1 & 0 \\ -s\beta & 0 & c\beta \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 \\ 0 & c\gamma & -s\gamma \\ 0 & s\gamma & c\gamma \end{bmatrix}。
$$

其中，$c\alpha=\cos\alpha$；$s\alpha=\sin\alpha$，$c\beta$，$s\beta$ 和 $c\gamma$，$s\gamma$ 依此类推。将矩阵相乘得

$$
{}^{A}_{B}R_{xyz}(\gamma,\beta,\alpha)=\begin{bmatrix} c\alpha\, c\beta & c\alpha\, s\beta\, s\gamma - s\alpha\, c\gamma & c\alpha\, s\beta\, c\gamma + s\alpha\, s\gamma \\ s\alpha\, c\beta & s\alpha\, s\beta\, s\gamma + c\alpha\, c\gamma & s\alpha\, s\beta\, c\gamma - c\alpha\, s\gamma \\ -s\beta & c\beta\, s\gamma & c\beta\, c\gamma \end{bmatrix}, \tag{2.37}
$$

-

现在来讨论它的逆问题(RPY 角反解)：从给定的旋转矩阵求出等价的绕固定轴 x-y-z 的转角 γ，β 和 α。令

$$ {}^{A}_{B}R_{zyx}(\gamma,\beta,\alpha)=\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}。 \tag{2.38} $$

这是一组超越方程，有 3 个未知数，共有 9 个方程，其中有 6 个方程不独立，因此可以利用其中的 3 个方程解出 3 个未知数。

从式(2.37)和(2.38)可以看出：

$$ \cos\beta=\sqrt{r_{11}^2+r_{21}^2}。 \tag{2.39} $$

如果 $\cos\beta\neq 0$，则得到各个角的反正切表达式：

$$ \left.\begin{aligned} \beta &= \mathrm{Atan2}(-r_{31},\sqrt{r_{11}^2+r_{21}^2}); \\ \alpha &= \mathrm{Atan2}(r_{21},r_{11}); \\ \gamma &= \mathrm{Atan2}(r_{32},r_{33})。 \end{aligned}\right\} \tag{2.40} $$

式中，$\mathrm{Atan2}(y,x)$是双变量反正切函数*。

式(2.39)中的根式运算有两个解，总是取$-90°\leqslant\beta\leqslant 90°$中的一个解。这样做通常可以定义方位的各种描述之间的一一对应关系。当然，有时也要计算出所有的解。

如果 $\beta=\pm 90°$，$\cos\beta=0$，则上述反解(2.40)退化。这时只可解出 α 与 γ 的和或差，通常选择 $\alpha=0°$，从而解出结果如下：

假若 $\beta=90°$，则

$$ \left.\begin{aligned} \beta &= 90°; \\ \alpha &= 0°; \\ \gamma &= \mathrm{Atan2}(r_{12},r_{22})。 \end{aligned}\right\} \tag{2.41} $$

假若 $\beta=-90°$，则

$$ \left.\begin{aligned} \beta &= -90°; \\ \alpha &= 0°; \\ \gamma &= -\mathrm{Atan2}(r_{12},r_{22})。 \end{aligned}\right\} \tag{2.42} $$

-

绕动轴ZYX旋转（Euler角）（内旋）

- 过程如下：B绕B的Z轴旋转α角，再绕B的Y轴旋转β角，最后绕B的X轴旋转γ角，完成旋转。整个过程，A不动B动。 旋转矩阵的计算方法如下：$R=R_Z*R_Y*R_X$。乘法顺序：从左向右

$$ \begin{aligned} {}^{A}_{B}R_{zyx}(\alpha,\beta,\gamma) &= R(z,\alpha)R(y,\beta)R(x,\gamma) \\ &= \begin{bmatrix} c\alpha & -s\alpha & 0 \\ s\alpha & c\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} c\beta & 0 & s\beta \\ 0 & 1 & 0 \\ -s\beta & 0 & c\beta \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 \\ 0 & c\gamma & -s\gamma \\ 0 & s\gamma & c\gamma \end{bmatrix}。 \end{aligned} \tag{2.43} $$

矩阵相乘后得

$$ {}^{A}_{B}R_{zyx}(\alpha,\beta,\gamma)=\begin{bmatrix} c\alpha\, c\beta & c\alpha\, s\beta\, s\gamma - s\alpha\, c\gamma & c\alpha\, s\beta\, c\gamma + s\alpha\, s\gamma \\ s\alpha\, c\beta & s\alpha\, s\beta\, s\gamma + c\alpha\, c\gamma & s\alpha\, s\beta\, c\gamma - c\alpha\, s\gamma \\ -s\beta & c\beta\, s\gamma & c\beta\, c\gamma \end{bmatrix}, \tag{2.44} $$

-

三、z-y-z 欧拉角

这种描述坐标系$\{B\}$的方位的法则如下：最初坐标系$\{B\}$与参考系$\{A\}$重合，首先使$\{B\}$绕z_B转α角，然后绕y_B转β角，最后绕z_B转γ角。

因为转动都是相对于运动坐标系$\{B\}$来描述的，又因为这三次转动的顺序是先绕z_B，再绕y_B，最后绕z_B，所以称为z-y-z欧拉角描述法。

根据"从左向右"的原则，可以求得与之等价的旋转矩阵：

$$
\begin{aligned}
{}^A_BR_{zyz}(\alpha,\beta,\gamma) &= R(z,\alpha)R(y,\beta)R(z,\gamma) \\
&= \begin{bmatrix} c\alpha\, c\beta\, c\gamma - s\alpha\, s\gamma & -c\alpha\, c\beta\, s\gamma - s\alpha\, c\gamma & c\alpha\, s\beta \\ s\alpha\, c\beta\, c\gamma + c\alpha\, s\gamma & -s\alpha\, c\beta\, s\gamma + c\alpha\, c\gamma & s\alpha\, s\beta \\ -s\beta\, c\gamma & s\beta\, s\gamma & c\beta \end{bmatrix}。
\end{aligned} \tag{2.45}
$$

反之，由旋转矩阵求解等价的z-y-z欧拉角的方法如下。令

$$
{}^A_BR_{zyz}(\alpha,\beta,\gamma) = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}, \tag{2.46}
$$

如果 $\sin\beta \neq 0$，则

$$
\left.\begin{aligned}
\beta &= \mathrm{Atan2}(\sqrt{r_{31}^2 + r_{32}^2}, r_{33}); \\
\alpha &= \mathrm{Atan2}(r_{23}, r_{13}); \\
\gamma &= \mathrm{Atan2}(r_{32}, -r_{31})。
\end{aligned}\right\} \tag{2.47}
$$

虽然 $\sin\beta = \sqrt{r_{31}^2 + r_{32}^2}$ 有两个解存在，但总是取 $0 \leqslant \beta \leqslant 180°$ 范围内的一个解。当 $\beta = 0°$ 或 $180°$ 时，解(2.47)是退化的，此时只能得到α与γ的和或差。通常选取$\alpha = 0°$，所得结果如下：

如果 $\beta = 0°$，则解为

$$
\left.\begin{aligned}
\beta &= 0°; \\
\alpha &= 0°; \\
\gamma &= \mathrm{Atan2}(-r_{12}, r_{11})。
\end{aligned}\right\} \tag{2.48}
$$

如果 $\beta = 180°$，则解为

$$
\left.\begin{aligned}
\beta &= 180°; \\
\alpha &= 0°; \\
\gamma &= \mathrm{Atan2}(r_{12}, -r_{11})。
\end{aligned}\right\} \tag{2.49}
$$

欧拉角的表示方式比较直观，但是有几个缺点：

- (1) 欧拉角的表示方式不唯一。给定某个起始朝向和目标朝向，即使给定yaw、pitch、roll的顺序，也可以通过不同的yaw/pitch/roll的角度组合来表示所需的旋转。比如，同样的yaw-pitch-roll顺序，(0,90,0)和(90,90,90)会将刚体转到相同的位置。这其实主要是由于万向锁(Gimbal Lock)

引起的 (2) 欧拉角的插值比较难。 (3) 计算旋转变换时，一般需要转换成旋转矩阵，这时候需要计算很多sin, cos，计算量较大。